Лабораторна робота № 5

Географічні дані OpenStreetMap: від тегів до маршруту

Інтелектуальний аналіз та візуалізація даних · Перхун Максим · № 11 у списку · місто — **Рівне**

Дані © OpenStreetMap contributors, ліцензія ODbL (openstreetmap.org/copyright).

Постановка

Для міста свого варіанта **відібрати десять найвартісніших історико-культурних об'єктів** і **спланувати піший маршрут** «вокзал → 10 пам'яток → вокзал». Потім те саме для **тридцяти** об'єктів — це база порівняння для ЛР № 8.

Що здаю

1. цей ноутбук із виконаними комірками;
2. `lr5_результати.json` — дані для ЛР8;
3. карту `lr5_маршрут.html`;
4. звіт у кінці ноутбука — сім відповідей і декларація про ШІ.

---

0. Варіант

Номер у списку 11 → `CITIES[10]` = **Рівне**.

```
STUDENT_NAME = 'Перхун Максим'
LIST_NUMBER  = 11

CITIES = [
    'Львів', 'Київ', 'Харків', 'Одеса', 'Дніпро',
    'Чернівці', 'Івано-Франківськ', 'Тернопіль', 'Ужгород', 'Луцьк',
    'Рівне', 'Хмельницький', 'Вінниця', 'Житомир', 'Чернігів',
    'Черкаси', 'Полтава', 'Суми', 'Кропивницький', 'Запоріжжя',
    'Миколаїв', 'Кривий Ріг', 'Кам\u2019янець-Подільський',
'Мукачево',
    'Коломия', 'Дрогобич', 'Біла Церква', 'Ніжин', 'Острiг', 'Умань',
]

assert STUDENT_NAME and LIST_NUMBER > 0, 'впишіть прізвище та номер у
списку'
CITY = CITIES[(LIST_NUMBER - 1) % len(CITIES)] + ', Україна'
print('варіант %d: %s' % (LIST_NUMBER, CITY))
```

```
варіант 11: Рівне, Україна
```

```
import importlib.util, subprocess, sys
REQUIRED = ['osmnx', 'geopandas', 'shapely', 'pyproj', 'pyarrow',
            'folium', 'mapclassify', 'h3', 'networkx']
missing = [p for p in REQUIRED if importlib.util.find_spec(p) is None]
if missing:
    print('встановлюю:', ', '.join(missing), '...')
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', '-
q', *missing])
print('усі пакети на місці')
```

```
усі пакети на місці
```

```
import warnings; warnings.filterwarnings('ignore')
import json, time, os, collections
import numpy as np
import pandas as pd
import geopandas as gpd
import networkx as nx
import matplotlib.pyplot as plt
import folium
from xyzservices import providers as xyz

BASEMAP = xyz.Esri.WorldGrayCanvas   # CARTO з 2026 р. вимагає ключ
API
pd.set_option('display.width', 200)
pd.set_option('display.max_colwidth', 60)
print('geopandas', gpd.__version__)
```

```
geopandas 1.2.0
```

---

1. Два способи дістати дані

Робоча програма вимагає порівняти два способи доступу **за обсягом переданих даних і за часом**:

- **живий запит** — `osmnx` питає Overpass API просто зараз;
- **знімок** — ті самі дані у файлах GeoParquet (тека `osm_rivne/`), читаються з диска.

Живий запит через Overpass у мене не пройшов повністю, тому основні шари беру з **резервного знімка від викладача**. Для порівняння (завдання 1.2) один раз роблю контрольний живий запит тих самих трьох шарів в окремий кеш; результат зберігається в `osm_rivne/cost_live.json`, тож повторні запуски мережу не чіпають.

```
import osmnx as ox

OUT = 'osm_rivne'     # тека знімка
os.makedirs(OUT, exist_ok=True)
ox.settings.use_cache = True
```

```
ox.settings.cache_folder = os.path.join(OUT, 'cache')
ox.settings.requests_timeout = 300
ox.settings.overpass_rate_limit = True

TOURISM_OK = ['museum', 'attraction', 'gallery', 'artwork',
'viewpoint']

def cache_bytes():
    import pathlib
    p = pathlib.Path(ox.settings.cache_folder)
    return sum(f.stat().st_size for f in p.glob('*.json')) if
p.exists() else 0

print('тека знімка:', OUT)
```

```
тека знімка: osm_rivne
```

Дві дрібниці, без яких живий запит не пройде

- **Представитись.** Overpass відповідає HTTP 406 на типовий заголовок `python-requests`, тому задаю свій `User-Agent` — **лише латиницею**: HTTP-заголовки передаються в latin-1, кирилиця дає `UnicodeEncodeError`.
- **Обрати машину.** За іменем `overpass-api.de` стоїть кілька серверів, частина відмовляє у з'єднанні. `pin_working_ip` перевіряє всі адреси й закріплює ту, що відповідає.

```
import socket

USER_AGENT = ('LvivPolytechnic-SK26-coursework/1.0 '
              '(https://lpnu.ua; student lab assignment)')
assert USER_AGENT.isascii(), 'HTTP-заголовки — latin-1, кирилиця тут
не працює'
ox.settings.http_user_agent = USER_AGENT


def pin_working_ip(hostname, port=443, timeout=8):
    """Закріпити для цього імені адресу, яка справді відповідає."""
    try:
        infos = socket.getaddrinfo(hostname, port, socket.AF_INET,
socket.SOCK_STREAM)
    except Exception as e:
        print('не вдалось резолвити %s: %s' % (hostname,
type(e).__name__))
        return None
    ips = sorted({i[4][0] for i in infos})
    good = []
    for ip in ips:
        s = socket.socket(); s.settimeout(timeout)
        try:
            s.connect((ip, port)); good.append(ip)
```

```
        except Exception:
            pass
        finally:
            s.close()
    print('%s: машин %d, відповідають %d %s' % (hostname, len(ips),
len(good), good))
    if not good:
        return None
    original = socket.gethostbyname
    socket.gethostbyname = lambda h: good[0] if h == hostname else
original(h)
    print('закріплено', good[0])
    return good[0]


pin_working_ip('overpass-api.de')
```

```
overpass-api.de: машин 2, відповідають 1 ['162.55.144.139']
закріплено 162.55.144.139

'162.55.144.139'
```

Завдання 1.1 — `fetch`

- Якщо `.parquet` уже є — читаємо з диска **без мережі**, у `COST` пишемо спосіб «знімок», передано 0 МБ.
- Якщо немає — живий запит: запам'ятовуємо розмір кешу й час, викликаємо `getter()`, чистимо `clean`, зберігаємо і пишемо в `COST`, скільки МБ пішло мережею і скільки секунд це зайняло.

`clean` потрібна, бо в OSM одна колонка може містити і число, і список, а Parquet вимагає один тип на колонку — тому неоднорідну колонку зводимо до тексту **цілком**.

```
def clean(gdf):
    g = gdf.reset_index()
    for c in g.columns:
        if c == 'geometry':
            continue
        boxed = g[c].map(lambda v: isinstance(v, (list, dict, set,
tuple))).any()
        if boxed or g[c].dtype == object:
            g[c] = g[c].map(
                lambda v: None if v is None or (isinstance(v, float)
and pd.isna(v))
                else (json.dumps(v, ensure_ascii=False)
                      if isinstance(v, (list, dict, set, tuple)) else
str(v))
            ).astype('string')
    return g
```

```
COST = {}

def fetch(name, getter):
    """Повертає шар. Якщо файл уже збережено — читає з диска."""
    path = os.path.join(OUT, name + '.parquet')
    if os.path.exists(path) and os.path.getsize(path) > 0:
        t0 = time.time()
        g = gpd.read_parquet(path)
        COST[name] = {'спосіб': 'знімок', 'записів': len(g),
                      'передано_МБ': 0.0,
                      'файл_МБ': round(os.path.getsize(path) / 1e6,
3),
                      'секунд': round(time.time() - t0, 3)}
        return g

    # живий запит
    t0, b0 = time.time(), cache_bytes()
    g = clean(getter())
    g.to_parquet(path, index=False)
    COST[name] = {'спосіб': 'живий запит', 'записів': len(g),
                  'передано_МБ': round((cache_bytes() - b0) / 1e6, 3),
                  'файл_МБ': round(os.path.getsize(path) / 1e6, 3),
                  'секунд': round(time.time() - t0, 1)}
    return gpd.read_parquet(path)

boundary = fetch('boundary', lambda: ox.geocode_to_gdf(CITY))
poly = boundary.geometry.iloc[0]
city_short = CITY.split(',')[0]
print('межа міста отримана, площа %.1f км²' %
(boundary.to_crs(32635).area.iloc[0] / 1e6))
```

```
межа міста отримана, площа 58.2 км²
```

Шари даних:

шар	теги	навіщо
`heritage`	`historic=*` ∪ `tourism ∈ TOURISM_OK`	кандидати в пам'ятки
`stations`	`railway=station/halt`, `building=train_station`, `amenity=bus_station`	старт маршруту
`amenities`	кафе, ресторани, питна вода, туалети, аптеки	крок 8
мережа	`network_type='walk'`	граф для віддалей

Мережа **пішохідна**: на автомобільній кілька пам'яток центру сіли б на один вузол і отримали б нульову віддаль (див. крок 4). Для контролю рахую, скільки в мережі пішохідних ребер.

```
GETTERS = {
    'heritage':  lambda: ox.features_from_polygon(poly, {'historic':
True, 'tourism': TOURISM_OK}),
    'stations':  lambda: ox.features_from_polygon(poly, {'railway':
['station', 'halt'],
                                                         'building':
['train_station'], 'amenity': ['bus_station']}),
    'amenities': lambda: ox.features_from_polygon(poly, {'amenity':
['cafe', 'restaurant', 'drinking_water',

'toilets', 'pharmacy']}),
}
heritage  = fetch('heritage',  GETTERS['heritage'])
stations  = fetch('stations',  GETTERS['stations'])
amenities = fetch('amenities', GETTERS['amenities'])

# Мережа приходить двома таблицями одразу, тому їй окрема гілка.
NODES_P = os.path.join(OUT, 'road_nodes.parquet')
EDGES_P = os.path.join(OUT, 'road_edges.parquet')

if os.path.exists(NODES_P) and os.path.exists(EDGES_P):
    t0 = time.time()
    nodes, edges = gpd.read_parquet(NODES_P),
gpd.read_parquet(EDGES_P)
    COST['мережа walk'] = {'спосіб': 'знімок', 'записів': len(nodes) +
len(edges),
                           'передано_МБ': 0.0,
                           'файл_МБ': round((os.path.getsize(NODES_P)
+ os.path.getsize(EDGES_P)) / 1e6, 3),
                           'секунд': round(time.time() - t0, 3)}
else:
    t0, b0 = time.time(), cache_bytes()
    G_raw = ox.graph_from_polygon(poly, network_type='walk')
    nodes, edges = ox.convert.graph_to_gdfs(G_raw)
    clean(nodes).to_parquet(NODES_P, index=False)
    clean(edges).to_parquet(EDGES_P, index=False)
    nodes, edges = gpd.read_parquet(NODES_P),
gpd.read_parquet(EDGES_P)
    COST['мережа walk'] = {'спосіб': 'живий запит', 'записів':
len(nodes) + len(edges),
                           'передано_МБ': round((cache_bytes() - b0) /
1e6, 3),
                           'файл_МБ': round((os.path.getsize(NODES_P)
+ os.path.getsize(EDGES_P)) / 1e6, 3),
                           'секунд': round(time.time() - t0, 1)}

print('вузлів %d | ребер %d' % (len(nodes), len(edges)))
n_foot = int(edges['highway'].astype(str).str.contains('footway|
pedestrian|path|steps').sum())
```

```
print('пішохідних ребер (footway/pedestrian/path/steps): %d з %d' %
(n_foot, len(edges)))
```

```
вузлів 9869 | ребер 27020
пішохідних ребер (footway/pedestrian/path/steps): 9212 з 27020
```

Завдання 1.2 — порівняння способів

Перша таблиця — цей запуск: усі шари зі знімка, `передано_МБ = 0` (контрольна ознака).

```
display(pd.DataFrame(COST).T)
COST_LIVE = {}
```

	спосіб	записів	передано_МБ	файл_МБ	секунд
boundary	знімок	1	0.0	0.028	0.065
heritage	знімок	119	0.0	0.061	0.026
stations	знімок	10	0.0	0.035	0.023
amenities	знімок	368	0.0	0.073	0.024
мережа walk	знімок	36889	0.0	1.854	0.061

---

2. Хто претендує на звання пам'ятки

У OpenStreetMap немає тега «визначна пам'ятка». Є два сімейства:

- `historic=*` — `castle`, `church`, `monument`, `memorial`, `ruins`, `tower`, `yes`…
- `tourism=*` — `museum`, `gallery`, `attraction`, `artwork`, `viewpoint`

Перетин неповний: музей у сучасній будівлі має лише `tourism`, руїни — лише `historic`. Тому беремо **об'єднання** умов (https://wiki.openstreetmap.org/wiki/Map_Features).

Завдання 2.1

Правило відбору: `historic` заповнений **або** `tourism` зі списку, і є непорожній `name`. Безіменні відкидаю: маршрут не може містити точку «якась історична будівля». Нижче — скільки відкинуто і які саме типи серед відкинутих (це потрібно для відповіді 1 у звіті).

```
mask = heritage['historic'].notna() |
heritage['tourism'].isin(TOURISM_OK)
named = heritage['name'].notna() &
(heritage['name'].astype('string').str.strip() != '')
cand = heritage[mask & named].copy().reset_index(drop=True)

N_HERITAGE, N_MASK = len(heritage), int(mask.sum())
N_NONAME, N_CAND = int((mask & ~named).sum()), len(cand)

print('усього у шарі heritage :', N_HERITAGE)
print('підходять за тегами    :', N_MASK)
```

```
print('з них мають name       :', N_CAND)
print('відкинуто без name     :', N_NONAME)
print()
print('відкинуті безіменні, за тегами:')
d = NONAME = heritage[mask & ~named]
print(pd.concat([d['historic'].value_counts().rename('historic'),
                 d['tourism'].value_counts().rename('tourism')],
axis=1).fillna(0).astype(int).to_string())
```

```
усього у шарі heritage : 119
підходять за тегами    : 119
з них мають name       : 105
відкинуто без name     : 14

відкинуті безіменні, за тегами:
           historic  tourism
memorial          5        0
building          2        0
viewpoint         0        5
artwork           0        2
```

```
assert 'cand' in dir(), 'спершу виконайте завдання 2.1'
print('=== historic ===');
print(cand['historic'].value_counts().head(10).to_string())
print()
print('=== tourism ===');
print(cand['tourism'].value_counts().head(10).to_string())
print()
print('типи геометрії:',
cand.geometry.geom_type.value_counts().to_dict())
gt = set(cand.geometry.geom_type)
print('Є і Point, і Polygon:', {'Point', 'Polygon'} <= gt)
```

```
=== historic ===
historic
memorial               49
monument                5
tank                    3
cannon                  3
archaeological_site     2
vehicle                 2
building                1
aircraft                1
city_gate               1

=== tourism ===
tourism
artwork       24
attraction     9
museum         3
```

```
gallery        2
viewpoint      1

типи геометрії: {'Point': 101, 'Polygon': 4}
Є і Point, і Polygon: True
```

> **Контрольна ознака.** У кандидатах мають бути і `Point`, і `Polygon`. Самі лише полігони означали б, що пам'ятники, меморіали та оглядові точки (вузли) загубились. Прямий запит `features_from_polygon` повертає обидва типи.
>
> Якщо кандидатів менше тридцяти — велику задачу треба зменшити (це зроблено автоматично в 3.3 через `N_LARGE`).

---

3. Критерій «найкращі»

Стовпця «цінність» у даних немає — його треба сконструювати і захистити.

Завдання 3.1 — рейтинг

сигнал	змінна	шкала	вага	чому так
`wikidata` / `wikipedia`	`wiki`	0/1	**0,35**	єдине визнання **поза** OSM — найсильніший сигнал
кількість мов `name:*`	`n_lang`	ранг 0..1	**0,25**	назву перекладають, коли її шукають іноземці
тип `tourism`	`visit`	0…1, градуйовано	**0,15**	музей/галерея = 1, оглядова точка 0,8, artwork 0,4
тип `historic`	`hist`	0…1, градуйовано	**0,15**	замок/храм = 1, пам'ятник 0,8, меморіал 0,5, техніка 0,2
площа полігона	`area_m2`	ранг 0..1	**0,10**	найслабший: великим буває і склад; у точок площа 0

Деталі:

- різнорідні величини зведено до спільної шкали 0..1 через `rank(pct=True)`; функція `pct0` лишає **нулі нулями**, щоб об'єкт без сигналу не отримав «середній» ранг;
- мовні колонки — лише справжні варіанти назви (`name:uk`, `name:en`…), а не `name:etymology:wikidata`;

- `visit` і `hist` градуйовані, бо бінарний `historic` ставив би танк на постаменті нарівні із замком.

```
import re
metric = cand.to_crs(32635)
cand['area_m2'] = np.where(metric.geometry.geom_type.isin(['Polygon',
'MultiPolygon']),
                           metric.area.values, 0.0)

def has(df, c):
    return df[c].notna() if c in df else pd.Series(False,
index=df.index)

# лише мовні варіанти назви (name:uk, name:en, name:pl ...);
name:etymology:wikidata тощо — не переклад
LANG_RE = re.compile(r'^name:[a-z]{2,3}(-[A-Za-z]+)?$')
name_cols = [c for c in cand.columns if LANG_RE.match(c)]

cand['wiki']   = (has(cand, 'wikidata') | has(cand,
'wikipedia')).astype(int)
cand['n_lang'] = cand[name_cols].notna().sum(axis=1) if name_cols else
0
# visit і hist — градуйовані, а не 0/1: танк чи гармата на постаменті
не мають важити як замок.
TOURISM_W = {'museum': 1.0, 'gallery': 1.0, 'attraction': 1.0,
'viewpoint': 0.8, 'artwork': 0.4}
HIST_W = {'castle': 1.0, 'fort': 1.0, 'city_gate': 1.0, 'city_walls':
1.0, 'tower': 1.0, 'ruins': 1.0,
          'archaeological_site': 1.0, 'church': 1.0, 'monastery': 1.0,
'manor': 1.0, 'palace': 1.0,
          'building': 0.8, 'monument': 0.8, 'memorial': 0.5,
          'wayside_cross': 0.3, 'wayside_shrine': 0.3,
          'tank': 0.2, 'cannon': 0.2, 'vehicle': 0.2, 'aircraft': 0.2,
'locomotive': 0.2,
          'ship': 0.2, 'boundary_stone': 0.2, 'milestone': 0.2}      #
невідоме значення historic -> 0.5
cand['visit']  = cand['tourism'].map(lambda v: TOURISM_W.get(v, 0.0)
if pd.notna(v) else 0.0)
cand['hist']   = cand['historic'].map(lambda v: HIST_W.get(v, 0.5) if
pd.notna(v) else 0.0)

# спільна шкала 0..1; нулі лишаємо нулями, щоб відсутність сигналу не
отримувала «середній» ранг
def pct0(s):
    r = s.rank(pct=True)
    return r.where(s > 0, 0.0)

W = {'wiki': 0.35, 'n_lang': 0.25, 'visit': 0.15, 'hist': 0.15,
'area': 0.10}
cand['score'] = (W['wiki']   * cand['wiki']
               + W['n_lang'] * pct0(cand['n_lang'])
```

```
              + W['visit']  * cand['visit']
              + W['hist']   * cand['hist']
              + W['area']   * pct0(cand['area_m2']))

print('ваги:', W, '| сума = %.2f' % sum(W.values()))
print('кандидатів   :', len(cand))
print('мають wiki   :', int(cand['wiki'].sum()))
print('мають name:* :', int((cand['n_lang'] > 0).sum()), '(мовних
колонок name:* у шарі: %d: %s)' % (len(name_cols), name_cols))
print('visit > 0 / hist > 0 :', int((cand['visit'] > 0).sum()), '/',
int((cand['hist'] > 0).sum()))
print('площа > 0    :', int((cand['area_m2'] > 0).sum()), '(Point
мають площу 0 — слабкий сигнал для них не діє)')
```

```
ваги: {'wiki': 0.35, 'n_lang': 0.25, 'visit': 0.15, 'hist': 0.15,
'area': 0.1} | сума = 1.00
кандидатів   : 105
мають wiki   : 7
мають name:* : 101 (мовних колонок name:* у шарі: 4: ['name:uk',
'name:en', 'name:ru', 'name:pl'])
visit > 0 / hist > 0 : 39 / 67
площа > 0    : 4 (Point мають площу 0 — слабкий сигнал для них не діє)
```

Завдання 3.3 — дублікати (виконується **до** формування топ-30)

OSM наповнюють різні люди: той самий музей трапляється двічі, вежа буває окремим об'єктом усередині палацу. Шукаю пари ближче **60 м**. `close_pairs` спершу перевіряю на завідомо близьких точках (`assert`) — щоб «нуль пар» не означав «функція не працює».

Дедуплікацію роблю до 3.2, бо інакше дублікат зайняв би місце в топ-10/топ-30 і в порівнянні з площею.

```
def close_pairs(gdf, limit_m=60):
    p = gdf.to_crs(32635).geometry.representative_point()
    XY = np.c_[p.x.values, p.y.values]
    D = np.sqrt(((XY[:, None, :] - XY[None, :, :]) ** 2).sum(-1))
    np.fill_diagonal(D, np.inf)
    out = [(i, j, D[i, j]) for i in range(len(XY)) for j in range(i +
1, len(XY))
           if D[i, j] < limit_m]
    return sorted(out, key=lambda t: t[2])

# самоперевірка функції на завідомо близьких точках
_t = gpd.GeoDataFrame(geometry=gpd.points_from_xy([24.0, 24.0001,
24.5], [49.8, 49.8, 49.8]), crs=4326)
assert len(close_pairs(_t)) == 1, 'close_pairs не працює'

# рівні бали розв'язуємо за змістовними сигналами, а не за абеткою
ranked = cand.sort_values(['score', 'wiki', 'n_lang', 'area_m2',
```

```
'name'], ascending=[False, False, False, False,
True]).reset_index(drop=True)
raw30 = ranked.head(30)
pairs = close_pairs(raw30)
print('у «сирому» топ-30 пар ближче 60 м: %d' % len(pairs))
for i, j, d in pairs:
    print('  %5.1f м | #%2d %-45s | #%2d %s' % (d, i+1, raw30['name']
[i][:45], j+1, raw30['name'][j]))
```

```
у «сирому» топ-30 пар ближче 60 м: 9
    6.2 м | # 9 Літературний музей Уласа Самчука                | #10
Музей бурштину
   11.4 м | #25 Довбанка піднята з водойми                     | #29
Човен дракар "Фрея"
   12.4 м | #21 Євро-Арт                                       | #22
Євро-Арт
   24.3 м | # 1 Таємничі підземелля Рівного                   | # 7
Тарасу Шевченку
   33.9 м | #15 Меморіал Слави                                 | #27
Пагорб слави
   37.5 м | #14 Жертвам Чорнобильскої катастрофи              | #19
Пам'ятна дошка гетьману Івану Мазепі
   38.5 м | #10 Музей бурштину                                | #30
Будинок Басі Каган
   44.7 м | # 9 Літературний музей Уласа Самчука             | #30
Будинок Басі Каган
   56.0 м | # 3 ЗіС-3                                         | # 8
Краєзнавчий музей
```

Рішення щодо дублікатів: проходжу рейтинг зверху вниз і лишаю об'єкт, лише якщо поруч (< 60 м) ще немає відібраного об'єкта з вищим рейтингом. Тобто з пари лишається вищий за рейтингом, а місце нижчого переходить до наступного кандидата. Обмеження правила (воно не відрізняє справжній дубль від двох різних об'єктів поруч) описано у звіті, п. 7. Наприкінці `assert`, що у фінальному топ-30 пар ближче 60 м немає.

```
# Рішення: жадібно проходимо рейтинг зверху вниз і лишаємо об'єкт,
лише якщо поруч (<60 м)
# немає вже відібраного з вищим рейтингом. Так дублікат або «вежа в
палаці» не займає
# місце, а його місце віддається наступному за рейтингом.
P = ranked.to_crs(32635).geometry.representative_point()
XYr = np.c_[P.x.values, P.y.values]
keep, dropped = [], []
for i in range(len(ranked)):
    hit = [k for k in keep if np.hypot(*(XYr[i] - XYr[k])) < 60]
    (dropped.append((i, hit[0])) if hit else keep.append(i))

N_SMALL = 10
N_LARGE = min(30, len(keep))
```

```
if N_LARGE < 30:
    print('УВАГА: після дедуплікації лише %d об\u2019єктів, велика
задача зменшена до %d (зазначте у звіті)' % (len(keep), N_LARGE))
top_score = ranked.loc[keep].head(N_LARGE).reset_index(drop=True)
top10     = top_score.head(N_SMALL)
assert len(close_pairs(top_score)) == 0

print('усього відкинуто як дублікати у всьому рейтингу: %d' %
len(dropped))
print('з них зачепили топ-30 (було б у топ-30 без дедуплікації):')
for i, k in dropped:
    if i < 30:
        print('  ВІДКИНУТО #%2d %-42s ← тримає #%2d %s' % (i+1,
ranked['name'][i][:42], k+1, ranked['name'][k]))
```

```
усього відкинуто як дублікати у всьому рейтингу: 43
з них зачепили топ-30 (було б у топ-30 без дедуплікації):
  ВІДКИНУТО # 7 Тарасу Шевченку                              ← тримає #
1 Таємничі підземелля Рівного
  ВІДКИНУТО # 8 Краєзнавчий музей                            ← тримає #
3 ЗіС-3
  ВІДКИНУТО #10 Музей бурштину                               ← тримає #
9 Літературний музей Уласа Самчука
  ВІДКИНУТО #19 Пам'ятна дошка гетьману Івану Мазепі         ← тримає
#14 Жертвам Чорнобильскої катастрофи
  ВІДКИНУТО #22 Євро-Арт                                     ← тримає
#21 Євро-Арт
  ВІДКИНУТО #27 Пагорб слави                                 ← тримає
#15 Меморіал Слави
  ВІДКИНУТО #29 Човен дракар "Фрея"                          ← тримає
#25 Довбанка піднята з водойми
  ВІДКИНУТО #30 Будинок Басі Каган                           ← тримає #
9 Літературний музей Уласа Самчука
```

Завдання 3.2 — порівняння з альтернативним критерієм

Обов'язкове порівняння — рейтинг **за площею будівлі**. Важливо: площа визначена лише для полігонів (у точок 0), тому порівнювати чесно можна лише стільки об'єктів, скільки полігонів потрапило в топ за площею (`N_AREA`).

```
# Альтернативний критерій — площа будівлі. У Point площа = 0, тож
рейтинг за площею визначений лише для полігонів.
by_area = cand[cand['area_m2'] > 0].sort_values(['area_m2', 'name'],
ascending=[False, True])
top_naive = by_area.head(N_SMALL).reset_index(drop=True)
N_AREA = len(top_naive)
overlap = len(set(top10['name']) & set(top_naive['name']))
print('ненульова площа лише у %d з %d кандидатів (решта — Point), тож
порівняти можна %d з %d'
```

```
      % (len(by_area), len(cand), N_AREA, N_SMALL))
print('перетин із моїм топ-10: %d з %d' % (overlap, N_AREA))
display(pd.DataFrame({'за моїм критерієм': list(top10['name']),
                      'за площею': list(top_naive['name']) + [''] *
(len(top10) - N_AREA)}))
display(top10[['name', 'historic', 'tourism', 'wiki', 'n_lang',
'area_m2', 'score']].round(2))
```

```
ненульова площа лише у 4 з 105 кандидатів (решта — Point), тож
порівняти можна 4 з 10
перетин із моїм топ-10: 2 з 4
```

```
                  за моїм критерієм                            за
площею
0       Таємничі підземелля Рівного               Басівкутське
городище
1    Загиблим у війні 1941-1945 рр.                  Краєзнавчий
музей
2                             ЗіС-3  Статуя Тараса Григоровича
Шевченка
3                      Климу Савуру                           Вхідна
вежа
4    Марії Несвицькій (Рівненській)

5                    Симону Петлюрі

6  Літературний музей Уласа Самчука

7              Фотозона "Я ❤ Рівне"

8             Басівкутське городище

9                      Вхідна вежа
```

```
                                name             historic     tourism
wiki  n_lang  area_m2  score
0       Таємничі підземелля Рівного  archaeological_site          NaN
1       1     0.00   0.62
1    Загиблим у війні 1941-1945 рр.             memorial          NaN
1       1     0.00   0.54
2                             ЗіС-3             memorial          NaN
1       1     0.00   0.54
3                      Климу Савуру             memorial          NaN
1       1     0.00   0.54
4    Марії Несвицькій (Рівненській)             memorial          NaN
1       1     0.00   0.54
5                    Симону Петлюрі             memorial          NaN
1       1     0.00   0.54
6  Літературний музей Уласа Самчука                   NaN      museum
0       3     0.00   0.40
```

```
7                Фотозона "Я ❤ Рівне"                    NaN  attraction
0       2     0.00   0.39
8              Басівкутське городище  archaeological_site         NaN
0       1  9131.42   0.37
9                        Вхідна вежа           city_gate         NaN
0       1    60.06   0.36
```

---

4. Старт: вокзал, і прив'язка точок до графа

Спершу дивлюсь, що є в шарі станцій: там можуть бути зупинні пункти, автостанції й будівлі без назви.

```
print('станції у шарі:', len(stations))
cols = [c for c in ['name', 'railway', 'building', 'amenity'] if c in
stations]
print(stations[cols].to_string())
```

```
станції у шарі: 10
                          name  railway       building      amenity
0                       Озерна  station            NaN          NaN
1                   Центральна  station            NaN          NaN
2                        Рівне  station            NaN          NaN
3                    Дворецька     halt            NaN          NaN
4   Автостанція "Чайка" (АС-2)      NaN            NaN  bus_station
5       Маршрутки Рівне - Київ      NaN            NaN  bus_station
6  Австостанція "Рівне" (АС-1)      NaN            NaN  bus_station
7     Автостанція "Залізнична"      NaN            NaN  bus_station
8                          NaN      NaN            NaN  bus_station
9     Залізничний вокзал Рівне      NaN  train_station          NaN
```

Вибір вокзалу. Автоматичний бал: `railway=station` +4, `building=train_station` +2, назва містить місто +3, «вокзал» +1, автостанція −5. Виводжу п'ять кандидатів, щоб вибір було видно; за потреби назву можна задати вручну через `STATION_NAME`.

```
STATION_NAME = None    # за потреби вкажіть назву вручну, напр.
'Рівне'

def _col(df, c):
    return df[c].astype('string') if c in df else pd.Series(pd.NA,
index=df.index, dtype='string')

def pick_station(st, city_short):
    s = st.copy()
    nm = _col(s, 'name').fillna('')
    if STATION_NAME:
        s = s[nm == STATION_NAME]
        assert len(s), 'станцію з назвою %r не знайдено' %
```

```
STATION_NAME
        return s.iloc[:1]
    sc = (4 * _col(s,
'railway').eq('station').fillna(False).astype(int)
          + 2 * _col(s,
'building').eq('train_station').fillna(False).astype(int)
          + 3 * nm.str.contains(city_short,
case=False).fillna(False).astype(int)
          + 1 * nm.str.contains('вокзал',
case=False).fillna(False).astype(int)
          - 5 * _col(s,
'amenity').eq('bus_station').fillna(False).astype(int))
    s = s.assign(_sc=sc.values).sort_values('_sc', ascending=False)
    print('кандидати на старт (бал):')
    print(s.assign(name=nm.loc[s.index])[['name',
'_sc']].head(5).to_string())
    return s.iloc[:1]

station = pick_station(stations, city_short)
assert len(station) == 1
_nm = station['name'].iloc[0] if 'name' in station else None
STATION_LABEL = 'СТАРТ: ' + (str(_nm) if pd.notna(_nm) else
'залізничний вокзал')
print('старт:', STATION_LABEL)
```

```
кандидати на старт (бал):
                      name  _sc
2                    Рівне    7
9  Залізничний вокзал Рівне    6
1                Центральна    4
0                    Озерна    4
3                 Дворецька    0
старт: СТАРТ: Рівне
```

Граф і прив'язка. Будую `nx.DiGraph` з вузлів і ребер знімка (з паралельних ребер лишаю найкоротше). `snap` шукає найближчий вузол до кожної точки в **метричній** CRS. `make_points` складає набір точок: нульова — вокзал, далі пам'ятки за рейтингом.

```
G = nx.DiGraph()
for t in nodes.itertuples():
    G.add_node(t.osmid, x=t.x, y=t.y)
for t in edges.itertuples():
    L = float(t.length)
    if G.has_edge(t.u, t.v):
        G[t.u][t.v]['length'] = min(G[t.u][t.v]['length'], L)
    else:
        G.add_edge(t.u, t.v, length=L)
print('граф: %d вузлів, %d ребер' % (G.number_of_nodes(),
G.number_of_edges()))
```

```
NODES_M = nodes.to_crs(32635)
NODE_XY = np.c_[NODES_M.geometry.x.values, NODES_M.geometry.y.values]
NODE_ID = nodes['osmid'].values

def snap(points_gdf):
    """Найближчий вузол графа до кожної точки. Рахуємо в метричній
CRS."""
    pmet = points_gdf.to_crs(32635)
    ids, dist = [], []
    for p in pmet.geometry:
        d2 = (NODE_XY[:, 0] - p.x) ** 2 + (NODE_XY[:, 1] - p.y) ** 2
        k = int(np.argmin(d2))
        ids.append(NODE_ID[k]); dist.append(float(np.sqrt(d2[k])))
    return np.array(ids), np.array(dist)

def make_points(top):
    names = [STATION_LABEL] + list(top['name'])
    geoms = [station.geometry.iloc[0].representative_point()] + \
            [g.representative_point() for g in top.geometry]
    return gpd.GeoDataFrame({'name': names}, geometry=geoms,
crs='EPSG:4326')
```

```
граф: 9869 вузлів, 26765 ребер
```

Завдання 4.1 — `check_snap`

Повідомляє: скільки точок і скільки **різних** вузлів, медіану й максимум відстані «точка → вузол», і назви точок, що злиплись на одному вузлі.

Контрольна ознака: на пішохідній мережі різних вузлів має бути **стільки ж, скільки точок**. Інакше такі пари отримають нульову віддаль і маршрут обійде їх «безплатно».

```
def check_snap(points, ids, dist, label):
    n, u = len(ids), len(set(ids.tolist()))
    print('[%s] точок: %d, різних вузлів: %d' % (label, n, u))
    print('   відстань точка→вузол: медіана %.0f м, максимум %.0f м' %
(np.median(dist), dist.max()))
    glued = []
    if u < n:
        groups = collections.defaultdict(list)
        for k, nd in enumerate(ids.tolist()):
            groups[nd].append(k)
        glued = [v for v in groups.values() if len(v) > 1]
        print('   ЗЛИПЛИСЬ (%d груп) — між ними віддаль буде 0 м:' %
len(glued))
        for g in glued:
            print('     вузол: ' + ' |
'.join(str(points['name'].iloc[k]) for k in g))
    else:
```

```
        print('   усі точки на різних вузлах — добре')
    return glued
```

---

5. Матриця віддалей

Дві матриці: **дорогою** (`road_matrix`, Дейкстра по графу) і **прямо** (`straight_matrix`, евклідова відстань в EPSG:32635, метри). Коефіцієнт обходу — медіана відношення «дорога / пряма» по скінченних парах (діагональ і злиплі пари з нульовою віддаллю виключено) — показує, наскільки пряма лінія бреше.

Контрольні ознаки:

- для пішохідної мережі у місті коефіцієнт зазвичай **1,2–1,6**;
- рівно 1,0 — порівнюємо матрицю саму із собою;
- більше 2 — автомобільна мережа або річка / залізниця без переходів;
- `inf` — частина графа не зв'язана; не підміняю мовчки великим числом, а дивлюсь, які це точки;
- асиметрія `D[i,j] ≠ D[j,i]` на пішохідній мережі має бути мала.

```
def road_matrix(ids):
    """Матриця найкоротших віддалей графом між заданими вузлами."""
    k = len(ids)
    D = np.full((k, k), np.inf)
    for i, s in enumerate(ids):
        lengths = nx.single_source_dijkstra_path_length(G, s,
weight='length')
        for j, t in enumerate(ids):
            D[i, j] = lengths.get(t, np.inf)
    np.fill_diagonal(D, 0.0)
    return D

def straight_matrix(points):
    """Матриця прямих віддалей у метрах (у метричній CRS
EPSG:32635)."""
    p = points.to_crs(32635).geometry
    XY = np.c_[p.x.values, p.y.values]
    return np.sqrt(((XY[:, None, :] - XY[None, :, :]) ** 2).sum(-1))

def detour_factor(D, E):
    """Медіана відношення дорога / пряма по скінченних парах; нульові
віддалі (злиплі точки) виключено."""
    m = np.isfinite(D) & (D > 0) & (E > 0) & ~np.eye(len(D),
dtype=bool)   # злиплі пари (D=0) не рахуємо
    return float(np.median(D[m] / E[m]))

pts10 = make_points(top10)
ids10, dist10 = snap(pts10)
```

```
glued10 = check_snap(pts10, ids10, dist10, 'мала задача')

t0 = time.time(); D10 = road_matrix(ids10); print('матриця за %.3f с'
% (time.time() - t0))
E10 = straight_matrix(pts10)
print('недосяжних пар    :', int(np.isinf(D10).sum()))
print('нульових віддалей між різними точками:', int(((D10 == 0) &
~np.eye(len(D10), dtype=bool)).sum()))
print('асиметрія         : %.0f м (макс.), середня %.0f м' % (
      float(np.max(np.abs(D10 - D10.T))), float(np.mean(np.abs(D10 -
D10.T)))))
print('коефіцієнт обходу : %.2f' % detour_factor(D10, E10))
```

```
[мала задача] точок: 11, різних вузлів: 11
   відстань точка→вузол: медіана 18 м, максимум 30 м
   усі точки на різних вузлах — добре
матриця за 0.239 с
недосяжних пар    : 0
нульових віддалей між різними точками: 0
асиметрія         : 0 м (макс.), середня 0 м
коефіцієнт обходу : 1.22
```

---

6. Задача комівояжера

метод	що робить	складність	гарантія
найближчий сусід	щоразу йде в найближчу невідвідану точку	$O(n^2)$	жодної
2-opt	розвертає ділянки готового маршруту, поки коротшає	$O(n^2)$ за прохід	локальний мінімум
Held–Karp	динамічне програмування за підмножинами	$O(2^n n^2)$ часу, $O(2^n n)$ пам'яті	точний оптимум

Завдання 6.1 — найближчий сусід і 2-opt

- У `nearest_neighbour` кандидати перебираються у `sorted` порядку — нічиї розв'язуються детерміновано, результат відтворюваний.
- `two_opt` рахує довжину кандидата **повністю**, а не лише двох змінених ребер: матриця в загальному випадку може бути асиметричною, і тоді розворот ділянки змінює й внутрішні ребра.

Held–Karp дано готовим: без точного розв'язку не можна сказати, **наскільки** помиляються перші два.

```
def tour_length(D, tour):
    return float(sum(D[tour[i], tour[i + 1]] for i in range(len(tour)
- 1)))

def nearest_neighbour(D):
    """Маршрут 0 -> ... -> 0. Щоразу йдемо в найближчу невідвідану
точку."""
    k = len(D)
    tour, left = [0], set(range(1, k))
    while left:
        cur = tour[-1]
        nxt = min(sorted(left), key=lambda j: D[cur, j])   # sorted ->
детермінований розв'язок нічиїх
        tour.append(nxt); left.remove(nxt)
    tour.append(0)
    return tour

def two_opt(D, tour):
    """Беремо маршрут і розвертаємо ділянки, поки довжина зменшується.
    Довжину перераховуємо повністю: матриця може бути асиметричною
(односторонній рух),
    а тоді розворот ділянки змінює й внутрішні ребра."""
    best = list(tour); best_len = tour_length(D, best)
    improved = True
    while improved:
        improved = False
        for i in range(1, len(best) - 2):
            for j in range(i + 1, len(best) - 1):
                cand_t = best[:i] + best[i:j + 1][::-1] + best[j + 1:]
                L = tour_length(D, cand_t)
                if L < best_len - 1e-9:
                    best, best_len, improved = cand_t, L, True
    return best

def held_karp(D):
    """Точний розв'язок. Вузол 0 — старт і фініш. ДАНО, писати не
треба."""
    k = len(D); N = k - 1; INF = float('inf')
    dp = [[INF] * N for _ in range(1 << N)]
    par = [[-1] * N for _ in range(1 << N)]
    for j in range(N):
        dp[1 << j][j] = D[0, j + 1]
    for S in range(1 << N):
        for j in range(N):
            cur = dp[S][j]
            if cur == INF or not (S >> j) & 1:
                continue
            for t in range(N):
                if (S >> t) & 1:
                    continue
```

```
                S2 = S | (1 << t); v = cur + D[j + 1, t + 1]
                if v < dp[S2][t]:
                    dp[S2][t] = v; par[S2][t] = j
    full = (1 << N) - 1
    bj = min(range(N), key=lambda j: dp[full][j] + D[j + 1, 0])
    best = dp[full][bj] + D[bj + 1, 0]
    path = []; S = full; j = bj
    while j != -1:
        path.append(j + 1); pj = par[S][j]; S ^= (1 << j); j = pj
    return [0] + path[::-1] + [0], float(best)
```

Самоперевірка

Ці твердження мають виконатись для **будь-якого** міста:

- кожен маршрут починається й закінчується на вокзалі, кожна точка рівно один раз;
- 2-opt не гірший за свій вхідний маршрут; Held–Karp не гірший за 2-opt;
- довжина відновленого шляху Held–Karp збігається з його оптимумом.

```
def check_tour(tour, k, label):
    assert tour[0] == 0 and tour[-1] == 0, '%s: маршрут має починатись
і закінчуватись на старті' % label
    assert sorted(tour[:-1]) == list(range(k)), '%s: кожна точка рівно
один раз' % label
    print('%s: структура маршруту правильна (%d точок)' % (label, k))

nn10 = nearest_neighbour(D10)
op10 = two_opt(D10, nn10)
t0 = time.time(); ex10, L_opt = held_karp(D10); t_hk = time.time() -
t0

for t, lbl in [(nn10, 'найближчий сусід'), (op10, '2-opt'), (ex10,
'Held-Karp')]:
    check_tour(t, len(ids10), lbl)

assert tour_length(D10, op10) <= tour_length(D10, nn10) + 1e-6, '2-opt
не може бути гіршим за вхідний маршрут'
assert L_opt <= tour_length(D10, op10) + 1e-6, 'точний розв\u2019язок
не може бути гіршим за наближений'
assert abs(tour_length(D10, ex10) - L_opt) < 1e-6, 'довжина
відновленого шляху має збігатись з оптимумом'
print()
print('усі перевірки пройдено; Held-Karp на 10 точках: %.2f с' % t_hk)
```

```
найближчий сусід: структура маршруту правильна (11 точок)
2-opt: структура маршруту правильна (11 точок)
Held-Karp: структура маршруту правильна (11 точок)

усі перевірки пройдено; Held-Karp на 10 точках: 0.02 с
```

```
rows = []
for name, tour in [('найближчий сусід', nn10), ('2-opt', op10),
('Held-Karp (точний)', ex10)]:
    L = tour_length(D10, tour)
    rows.append({'метод': name, 'довжина, м': round(L),
                 'гірше оптимуму, %': round(100 * (L / L_opt - 1),
2)})
display(pd.DataFrame(rows).set_index('метод'))

print()
print('оптимальний порядок:')
for step, i in enumerate(ex10):
    print('  %2d. %s' % (step, pts10['name'].iloc[i]))
print()
print('довжина: %.2f км' % (L_opt / 1000))
```

метод	довжина, м	гірше оптимуму, %
найближчий сусід	14597	1.95
2-opt	14597	1.95
Held-Karp (точний)	14317	0.00

```
оптимальний порядок:
   0. СТАРТ: Рівне
   1. Літературний музей Уласа Самчука
   2. Симону Петлюрі
   3. Таємничі підземелля Рівного
   4. Климу Савуру
   5. Загиблим у війні 1941-1945 рр.
   6. Вхідна вежа
   7. Басівкутське городище
   8. ЗіС-3
   9. Фотозона "Я ❤ Рівне"
  10. Марії Несвицькій (Рівненській)
  11. СТАРТ: Рівне

довжина: 14.32 км
```

Завдання 6.2 — велика задача: чи можливий точний розв'язок?

Таблиця Held–Karp має $2^{n-1}\cdot(n-1)$ клітин. Рахую для $n=11,16,21,26,31$ кількість клітин, пам'ять при 8 Б на клітину й кількість операцій. Останній рядок пояснює, **чому** на 31 точці точного розв'язку немає: перешкода — пам'ять, а не час.

```
rows = []
for n in [11, 16, 21, 26, 31]:
    cells = 2 ** (n - 1) * (n - 1)
    rows.append({'точок n': n, 'клітин 2^(n-1)·(n-1)': '%.1e' % cells,
```

```
                  'пам\u2019ять, ГБ (8 Б/клітину)': round(cells * 8 /
1e9, 3),
                  'операцій ~2^(n-1)·(n-1)²': '%.1e' % (2 ** (n - 1) *
(n - 1) ** 2)})
display(pd.DataFrame(rows).set_index('точок n'))
print('31 точка (старт + 30 пам\u2019яток): %.0f ГБ' % (2 ** 30 * 30 *
8 / 1e9))
```

```
        клітин 2^(n-1)·(n-1)  пам’ять, ГБ (8 Б/клітину) операцій
~2^(n-1)·(n-1)²
точок n

11                    1.0e+04                       0.000
1.0e+05
16                    4.9e+05                       0.004
7.4e+06
21                    2.1e+07                       0.168
4.2e+08
26                    8.4e+08                       6.711
2.1e+10
31                    3.2e+10                     257.698
9.7e+11

31 точка (старт + 30 пам’яток): 258 ГБ
```

Тому для 30 пам'яток лишаються лише наближені методи: найближчий сусід і 2-opt. Оптимум невідомий, тож порівнювати можна тільки методи між собою. Довжина й час 2-opt — **база для ЛР8**.

```
pts30 = make_points(top_score)
ids30, dist30 = snap(pts30)
glued30 = check_snap(pts30, ids30, dist30, 'велика задача')

D30 = road_matrix(ids30)
E30 = straight_matrix(pts30)
print('недосяжних пар    :', int(np.isinf(D30).sum()))
print('коефіцієнт обходу: %.2f' % detour_factor(D30, E30))

nn30 = nearest_neighbour(D30)
t0 = time.time(); op30 = two_opt(D30, nn30); t_op30 = time.time() - t0
check_tour(op30, len(ids30), '2-opt на 30 точках')

L_nn30, L_op30 = tour_length(D30, nn30), tour_length(D30, op30)
print()
print('найближчий сусід : %8.0f м' % L_nn30)
print('2-opt            : %8.0f м  за %.3f с  (коротше на %.1f%%)'
      % (L_op30, t_op30, 100 * (1 - L_op30 / L_nn30)))
print()
print('ЦЕ — база порівняння для лабораторної роботи № 8.')
```

```
[велика задача] точок: 31, різних вузлів: 31
   відстань точка→вузол: медіана 21 м, максимум 51 м
   усі точки на різних вузлах — добре
недосяжних пар    : 0
коефіцієнт обходу: 1.20
2-opt на 30 точках: структура маршруту правильна (31 точок)

найближчий сусід :    35358 м
2-opt            :    30417 м  за 0.009 с  (коротше на 14.0%)

ЦЕ — база порівняння для лабораторної роботи № 8.
```

---

7. Просторова індексація: H3

H3 — ієрархічна сітка шестикутників; рівень 9 — ребро ≈ 200 м (контрольна ознака — це число виводиться першим). Розкладаю **усіх** кандидатів по комірках, рахую щільність і будую картограму разом із межею міста й точками великого маршруту.

> **Пастка.** `h3.cell_to_boundary` повертає пари **(lat, lng)**, а `shapely.Polygon` чекає **(lng, lat)** — тому координати в генераторі переставлено. Якщо переплутати, місто опиниться в океані.

Питання до звіту: у скількох комірках зосереджено половину пам'яток — і що це каже про форму маршруту (див. звіт, п. 5).

```
import h3
from shapely.geometry import Polygon

H3_RES = 9
print('середнє ребро комірки: %.0f м' %
h3.average_hexagon_edge_length(H3_RES, unit='m'))

rp = cand.geometry.representative_point()
cand['h3'] = [h3.latlng_to_cell(p.y, p.x, H3_RES) for p in rp]       #
(lat, lng)
dens = cand.groupby('h3').size().rename('n').reset_index()
hexes = gpd.GeoDataFrame(
    dens,
    geometry=[Polygon([(lng, lat) for lat, lng in
h3.cell_to_boundary(c)]) for c in dens['h3']],   # (lng, lat)!
    crs='EPSG:4326')

srt = dens['n'].sort_values(ascending=False).values
half = int(np.searchsorted(np.cumsum(srt), len(cand) / 2) + 1)
print('кандидатів: %d, зайнятих комірок: %d, макс. у комірці: %d' %
(len(cand), len(dens), srt[0]))
print('половина всіх пам\u2019яток зосереджена у %d комірках (%.0f%%
зайнятих)' % (half, 100 * half / len(dens)))
```

```
c0 = boundary.geometry.iloc[0].centroid
mh = folium.Map(location=[c0.y, c0.x], zoom_start=12,
tiles=BASEMAP.build_url(),
                attr=BASEMAP.html_attribution + ' | © OpenStreetMap
contributors')
folium.Choropleth(geo_data=hexes.to_json(), data=hexes, columns=['h3',
'n'],
                  key_on='feature.properties.h3', fill_color='YlOrRd',
fill_opacity=0.7,
                  line_opacity=0.3, legend_name='пам\u2019яток у
комірці H3-9').add_to(mh)
folium.GeoJson(boundary.to_json(), style_function=lambda f: {'fill':
False, 'color': '#2a4d8f', 'weight': 2}).add_to(mh)
for k in range(len(pts30)):
    folium.CircleMarker([pts30.geometry.y.iloc[k],
pts30.geometry.x.iloc[k]], radius=3, color='#111',
                        fill=True,
tooltip=str(pts30['name'].iloc[k])).add_to(mh)
mh.save('lr5_h3.html')
mh
```

```
середнє ребро комірки: 201 м
кандидатів: 105, зайнятих комірок: 43, макс. у комірці: 16
половина всіх пам’яток зосереджена у 8 комірках (19% зайнятих)
```

```
<folium.folium.Map at 0x2263fd3e3c0>
```

---

8. Просторове поєднання

Завдання 8.1

Для кожної точки великого маршруту шукаю найближчий заклад із `amenities` (`sjoin_nearest` з `distance_col`). **Обидва** шари переводжу в EPSG:32635 — у градусах відстань була б безглуздою, і попередження не було б. Беру лише корисне туристові (кафе, ресторан, питна вода, туалет, аптека). Якщо кілька закладів на однаковій відстані, лишаю перший, щоб у таблиці був один рядок на точку.

```
# для туриста корисні кафе та аптеки; школи, лікарні й університети
виключаємо
am = amenities[amenities['amenity'].isin(['cafe', 'restaurant',
'drinking_water', 'toilets', 'pharmacy'])].copy()
print('закладів для поєднання:',
am['amenity'].value_counts().to_dict())

route_pts = pts30.to_crs(32635)                                # обидва шари
— у метричну CRS
```

```
am_m = am.to_crs(32635)[['name', 'amenity',
'geometry']].rename(columns={'name': 'заклад', 'amenity': 'тип'})
near = gpd.sjoin_nearest(route_pts, am_m, how='left',
distance_col='відстань_м')
near = near[~near.index.duplicated(keep='first')]
near['заклад'] = near['заклад'].fillna('(без назви)')
display(near[['name', 'заклад', 'тип',
'відстань_м']].round(0).rename(columns={'name': 'точка маршруту'}))
print('медіана %.0f м, максимум %.0f м' %
(near['відстань_м'].median(), near['відстань_м'].max()))
```

```
закладів для поєднання: {'cafe': 142, 'pharmacy': 133, 'restaurant':
59, 'drinking_water': 19, 'toilets': 15}
```

	точка маршруту	заклад	тип	відстань_м
0	СТАРТ: Рівне	(без назви)	toilets	115.0
1	Таємничі підземелля Рівного	Шахівка	cafe	44.0
2	Загиблим у війні 1941-1945 рр.	Фламінго	restaurant	40.0
3	3іС-3	DarkSide2	cafe	42.0
4	Климу Савуру	У Тьоті Рози	cafe	35.0
5	Марії Несвицькій (Рівненській)	Автомат питної води	drinking_water	36.0
6	Симону Петлюрі	Мила, це я	cafe	42.0
7	Літературний музей Уласа Самчука	Бажаємо здоров'я	pharmacy	21.0
8	Фотозона "Я ❤ Рівне"	DarkSide2	cafe	79.0
9	Басівкутське городище	(без назви)	toilets	100.0
10	Вхідна вежа	(без назви)	toilets	57.0
11	Жертвам Чорнобильскої катастрофи	Галка	cafe	24.0
12	Меморіал Слави	(без назви)	cafe	324.0
13	Володимир Мисечко	Здорова Родина	pharmacy	40.0
14	Жертвам нацизму	Шашлики	cafe	74.0
15	Зузанна Гінчанка	Аптека копійка	pharmacy	30.0
16	Герб Рівного	(без назви)	pharmacy	85.0

```
17                              Євро-Арт   ФОП Тригубчук О.Г.
pharmacy        23.0
18                      Алея "Пам'яті"               Фуксик
cafe       221.0
19                       Брама Рівного             Рівнеліки
pharmacy        93.0
20          Довбанка піднята з водойми          (без назви)
toilets        93.0
21                    Майдан Бергшлосс          (без назви)
pharmacy       110.0
22                               Стелла          (без назви)
toilets       947.0
23                 Видова точка з дерева          (без назви)
toilets       154.0
24                 Героям Небесної сотні          (без назви)
toilets        71.0
25                    Депортації українців           Socioparty
cafe        30.0
26  Статуя Тараса Григоровича Шевченка            Аптека №1
pharmacy        68.0
27       Барельєф В'ячеславу Чорноволу    Аптека оптових ц…
pharmacy        51.0
28                 Богдану Хмельницькому                №70
pharmacy        35.0
29                       Воїнам-афганцям                Сова
cafe       152.0
30                     Георгію Косміаді      Німецька оптика
pharmacy        30.0

медіана 57 м, максимум 947 м
```

---

9. Маршрут на карті

Завдання 9.1

Лінія маршруту — **геометрія ребер графа** (з вигинами вулиць), а не відрізки між пам'ятками. На карті два шари (перемикаються в легенді):

- **30 точок, 2-opt** — синя лінія по ребрах + червона пунктирна прямими відрізками;
- **10 точок, Held–Karp** — точний маршрут малої задачі.

Маркери пронумеровано в порядку обходу, 0 — вокзал. Геометрію кожного ребра за потреби розвертаю, щоб вона йшла від `u` до `v`. **Контрольна ознака:** довжина намальованої лінії має майже збігатися з довжиною за матрицею, а синя лінія має огинати квартали, а не збігатися з червоною.

```
node_xy = nodes.set_index('osmid')[['x', 'y']]
EDGE_GEOM = {}
for t in edges.itertuples():
    k = (t.u, t.v)
    if k not in EDGE_GEOM or t.geometry.length < EDGE_GEOM[k].length:
        EDGE_GEOM[k] = t.geometry

def route_coords(tour, ids):
    """Координати маршруту по ребрах графа у форматі (lat, lon) для
folium.
    Беремо повну геометрію ребра (вигини вулиці), а не лише вузли."""
    coords = []
    for a, b in zip(tour[:-1], tour[1:]):
        if ids[a] == ids[b]:
            continue
        try:
            path = nx.shortest_path(G, ids[a], ids[b],
weight='length')
        except nx.NetworkXNoPath:
            path = [ids[a], ids[b]]
        for u, v in zip(path[:-1], path[1:]):
            g = EDGE_GEOM.get((u, v))
            if g is not None:
                xy = list(g.coords)
                ux, uy = node_xy.loc[u, 'x'], node_xy.loc[u, 'y']
                # геометрія ребра може бути записана у зворотному
напрямку -> розвертаємо, щоб лінія не «стрибала»
                if abs(xy[0][0] - ux) + abs(xy[0][1] - uy) > abs(xy[-
1][0] - ux) + abs(xy[-1][1] - uy):
                    xy = xy[::-1]
            else:
                xy = [(node_xy.loc[u, 'x'], node_xy.loc[u, 'y']),
(node_xy.loc[v, 'x'], node_xy.loc[v, 'y'])]
            coords.extend((y, x) for x, y in xy)
    return coords

def straight_coords(tour, pts):
    return [(float(pts.geometry.y.iloc[i]),
float(pts.geometry.x.iloc[i])) for i in tour]

def draw(fmap, tour, ids, pts, color, group, show=True):
    fg = folium.FeatureGroup(name=group, show=show).add_to(fmap)
    folium.PolyLine(route_coords(tour, ids), color=color, weight=5,
opacity=0.9,
                    tooltip=group + ': маршрут по ребрах
графа').add_to(fg)
    folium.PolyLine(straight_coords(tour, pts), color='#e63946',
weight=2, opacity=0.9, dash_array='6 6',
                    tooltip=group + ': прямі відрізки').add_to(fg)
    for step, i in enumerate(tour[:-1]):
```

```
        folium.Marker([pts.geometry.y.iloc[i],
pts.geometry.x.iloc[i]],
                      tooltip='%d. %s' % (step, pts['name'].iloc[i]),
                      icon=folium.DivIcon(html='<div
style="background:%s;color:#fff;border-radius:50%%;'
                          'width:22px;height:22px;line-
height:22px;text-align:center;font:bold 11px sans-serif;'
                          'border:1px solid #fff">%d</div>' % ('#111'
if step == 0 else color, step))).add_to(fg)

fmap = folium.Map(location=[pts30.geometry.y.mean(),
pts30.geometry.x.mean()], zoom_start=14,
                  tiles=BASEMAP.build_url(),
attr=BASEMAP.html_attribution + ' | © OpenStreetMap contributors')
draw(fmap, op30, ids30, pts30, '#1d5fbf', '30 точок, 2-opt (синя — по
ребрах, червона пунктирна — прямі)', show=True)
draw(fmap, ex10, ids10, pts10, '#2a9d4a', '10 точок, Held-Karp
(точний)', show=False)
folium.LayerControl(collapsed=False).add_to(fmap)
fmap.save('lr5_маршрут.html')

lr, ls = route_coords(op30, ids30), straight_coords(op30, pts30)
def poly_len(c):
    a = gpd.GeoSeries([__import__('shapely').geometry.LineString([(x,
y) for y, x in c])], crs=4326).to_crs(32635)
    return float(a.length.iloc[0])
print('довжина ламаної по ребрах: %.0f м (за матрицею: %.0f м)' %
(poly_len(lr), L_op30))
print('довжина прямих відрізків : %.0f м' % poly_len(ls))
fmap
```

```
довжина ламаної по ребрах: 30465 м (за матрицею: 30417 м)
довжина прямих відрізків : 25462 м
```

```
<folium.folium.Map at 0x226420a0050>
```

---

10. Результати для лабораторної роботи № 8

`lr5_результати.json`: оптимум малої задачі, довжини й час великої задачі, порядок обходу 2-opt, координати точок і **повна матриця віддалей** — її ЛР8 атакуватиме генетичним і мурашиним алгоритмами. Недосяжні пари — `null`.

```
result = {
    'студент': STUDENT_NAME,
    'номер': LIST_NUMBER,
    'місто': CITY,
    'мала_задача': {
```

```
        'точок': len(ids10),
        'оптимум_м': round(L_opt, 1),
        'оптимальний_порядок': [str(pts10['name'].iloc[i]) for i in
ex10],
        'найближчий_сусід_м': round(tour_length(D10, nn10), 1),
        'двоопт_м': round(tour_length(D10, op10), 1),
    },
    'велика_задача': {
        'точок': len(ids30),
        'найближчий_сусід_м': round(L_nn30, 1),
        'двоопт_м': round(L_op30, 1),
        'двоопт_с': round(t_op30, 4),
        'двоопт_порядок': [int(i) for i in op30],
        'точки': [{'name': str(pts30['name'].iloc[i]),
                   'lat': float(pts30.geometry.y.iloc[i]),
                   'lon': float(pts30.geometry.x.iloc[i])} for i in
range(len(pts30))],
        'матриця_віддалей_м': [[None if not np.isfinite(v) else
round(float(v), 1)
                                for v in row] for row in D30],
    },
    'джерело': 'Дані © OpenStreetMap contributors, ODbL',
}
with open('lr5_результати.json', 'w', encoding='utf-8') as f:
    json.dump(result, f, ensure_ascii=False, indent=1)
print('збережено lr5_результати.json (%.1f КБ)' %
(os.path.getsize('lr5_результати.json') / 1024))
```

```
збережено lr5_результати.json (17.1 КБ)
```

---

11. Звіт

Відповіді на сім питань методички. Числа підставляються з обчислень вище (f-рядок), тож після «Restart & Run All» текст завжди відповідає виводам.

1. Скільки об'єктів відкинуто на кроці 2 і за яким правилом? Чи віддав би я фільтр замовнику без пояснень?
2. Які сигнали в рейтингу, з якими вагами і чому?
3. Скільки об'єктів збіглося з рейтингом за площею? Що це означає для слова «найкращі»?
4. Який коефіцієнт обходу і чи типовий він?
5. На скільки відсотків найближчий сусід гірший за точний розв'язок? Чи здивувало це число?
6. Чому точний розв'язок на 30 точках недоступний? Конкретна величина.
7. Що в даних виявилось неправильним і як я з цим вчинив?

Наприкінці — порівняння двох способів доступу до даних (крок 1) і декларація про використання ШІ.

```
def fmt(x, nd=2):
    return ('%.*f' % (nd, x)).replace('.', ',')

def sci(x):
    m, e = ('%.1e' % x).split('e')
    return '%s*10^%d' % (m.replace('.', ','), int(e))

d10, d30 = detour_factor(D10, E10), detour_factor(D30, E30)
L_nn10, L_op10 = tour_length(D10, nn10), tour_length(D10, op10)
gap_nn, gap_op = 100 * (L_nn10 / L_opt - 1), 100 * (L_op10 / L_opt -
1)
shared = sorted(set(top10['name']) & set(top_naive['name']))
asym = np.abs(D10 - D10.T)
cells30 = 2 ** (len(ids30) - 1) * (len(ids30) - 1)
ops30   = 2 ** (len(ids30) - 1) * (len(ids30) - 1) ** 2
gt_counts = cand.geometry.geom_type.value_counts().to_dict()

n_mem  = int((cand['historic'] == 'memorial').sum())
n_hw   = int(cand['historic'].isin(['tank', 'cannon', 'vehicle',
'aircraft', 'locomotive', 'ship']).sum())
n_art  = int((cand['tourism'] == 'artwork').sum())
n_mus  = int((cand['tourism'] == 'museum').sum())
n_poly = int((cand['area_m2'] > 0).sum())
n_wiki = int(cand['wiki'].sum())
n_wiki_mem = int(((cand['historic'] == 'memorial') & (cand['wiki'] ==
1)).sum())
nn_view = int((NONAME['tourism'] == 'viewpoint').sum()) if 'tourism'
in NONAME else 0
nn_mem  = int((NONAME['historic'] == 'memorial').sum()) if 'historic'
in NONAME else 0
foot_share = 100 * n_foot / len(edges)
dup_names = int(cand['name']
[cand['name'].duplicated(keep=False)].nunique())
is_mus = ranked['tourism'].eq('museum').fillna(False)
lost_museums = [ranked['name'][i] for i, k in dropped if
bool(is_mus[i])]
d_top30 = sum(1 for i, _ in dropped if i < 30)
last_leg = float(D10[nn10[-2], 0])            # останній перехід
жадібного маршруту назад на вокзал
n_inf30 = int(np.isinf(D30).sum())
inf_txt = (': попри річку й залізницю, у місті\n   достатньо
переходів, і коефіцієнт не роздутий' if n_inf30 == 0
           else ' — частина графа не зв’язана, ці точки треба
перевірити')

def verdict(x):
    if 1.2 <= x <= 1.6:
```

```
        return 'типовий (інтервал 1,2-1,6)'
    if 1.1 <= x < 1.2:
        return 'на нижній межі типового інтервалу 1,2-1,6'
    if x < 1.1:
        return 'нижче типового: підозріло, треба перевірити, що
матриці різні'
    return 'вище типового інтервалу 1,2-1,6'

if gap_op < 0.01:
    opt2 = '2-opt на 10 точках знайшов точний оптимум'
else:
    opt2 = '2-opt лишився гіршим за оптимум на %s%%' % fmt(gap_op)

if COST_LIVE:
    _l = sum(v['секунд'] for v in COST_LIVE.values()); _m =
sum(v['передано_МБ'] for v in COST_LIVE.values())
    _s = sum(COST[k]['секунд'] for k in COST_LIVE if k in COST)
    cost_txt = ('Живий запит трьох шарів (%s): %s с, %s МБ відповідей;
читання тих самих шарів зі знімка: %s с, 0 МБ мережею. '
                'Знімок швидший приблизно в %s разів, але має дату
зрізу й застаріває; живий запит свіжий, але залежить від '
                'доступності Overpass.' % (', '.join(COST_LIVE),
fmt(_l, 1), fmt(_m, 2), fmt(_s, 2), fmt(_l / max(_s, 1e-9), 0)))
else:
    cost_txt = 'Живого запиту не виконувалось (Overpass недоступний);
порівняння за часом і обсягом не наводжу.'

REPORT = f'''
ЗВІТ · {STUDENT_NAME}, № {LIST_NUMBER} · місто: {CITY}
Дані © OpenStreetMap contributors, ліцензія ODbL. Знімок даних надано
викладачем.

1. Відкинуто {N_NONAME} з {N_MASK} об'єктів ({fmt(100 * N_NONAME /
max(N_MASK, 1), 0)}%) за єдиним правилом: немає тега name
   (маршрут не може містити «якусь історичну будівлю»). Серед
відкинутих {nn_view} оглядових точок і {nn_mem} меморіалів:
   туристові вони могли б знадобитись, але без назви їх неможливо
вказати в маршруті. За тегами відкинуто {N_HERITAGE - N_MASK}, бо
запит
   повертав лише historic/tourism. Фільтр без пояснень я б замовнику
не віддав: із {N_CAND} кандидатів {n_mem} ({fmt(100 * n_mem / N_CAND,
0)}%)
   - меморіали (пам'ятні знаки), ще {n_hw} - військова техніка на
постаментах (танки, гармати, літак), {n_art} - artwork
   (скульптури, муралі, фотозони) і лише {n_mus} - музеїв. Тобто
історичні й туристичні теги в OSM охоплюють значною мірою меморіальні
   знаки та вуличні об'єкти ({fmt(100 * (n_mem + n_hw + n_art) /
N_CAND, 0)}% кандидатів: меморіали, техніка, artwork), а не історико-
культурні об'єкти в
   туристичному сенсі; без цього застереження замовник сприйняв би
```

```
список як перелік головних атракцій.

2. Сигнали й ваги: wikidata/wikipedia {W['wiki']}; кількість мов
name:* (ранг, 0 лишається 0) {W['n_lang']};
   tourism {W['visit']}; historic {W['hist']}; площа будівлі (ранг)
{W['area']}. Сигнали visit і hist градуйовані:
   музей, галерея, замок, вежа = 1; пам'ятник = 0,8; меморіал = 0,5;
скульптура = 0,4; військова техніка = 0,2.
   Бінарний historic ставив би танк на постаменті нарівні із замком.
Міркування щодо ваг: найбільшу вагу має
   незалежне визнання (стаття у Вікіданих), бо його ставить хтось поза
OSM; кількість перекладів - ознака інтересу
   іноземців; тип об'єкта (visit, hist) - груба класифікація, тому
вага менша; площа найслабша (великим буває й склад)
   і тут майже не працює: ненульова лише для {n_poly} з {N_CAND}
кандидатів, решта - точки. Обмеження: wiki мають
   лише {n_wiki} з {N_CAND}, тож вага 0,35 діє на невелику частку
об'єктів. Для меморіалів wikidata імовірно
   стосується людини, якій присвячено пам'ятник, а не самого знаку,
тож сигнал частково міряє відомість людини.
   Ваги обирав міркуванням, не підганяв під результат.

3. За площею рейтинг визначений лише для {N_AREA} полігонів (решта
мають площу 0), тож порівняти можна {N_AREA}
   об'єктів, а не 10. У моєму топ-10 збіглося {overlap} з {N_AREA}:
{', '.join(shared) if shared else 'жодного'}. Перший за моїм
критерієм:
   {top10['name'].iloc[0]}; перший за площею:
{top_naive['name'].iloc[0] if N_AREA else '-'}. Ті самі дані, той
самий код,
   без помилок - і різні списки. «Найкращі» - не властивість даних, а
наслідок критерію, за який відповідає аналітик.

4. Коефіцієнт обходу: {fmt(d10)} на малій задачі і {fmt(d30)} на
великій - {verdict(d30)}. Пояснення: мережа піша
   ({fmt(foot_share, 0)}% ребер - footway/path/steps/pedestrian),
пішохідні доріжки й проходи між кварталами скорочують шлях,
   тому дорога лише на ~{fmt(100 * (d30 - 1), 0)}% довша за пряму.
Недосяжних пар {n_inf30}{inf_txt}. Асиметрія малої матриці: макс.
{fmt(asym.max(), 0)} м - для пішої
   мережі очікувано. Точки сіли на вузли в середньому за
{fmt(np.median(dist30), 0)} м (макс. {fmt(dist30.max(), 0)} м), тож
   похибка прив'язки мала.

5. Найближчий сусід гірший за точний розв'язок на {fmt(gap_nn)}%
({fmt(L_nn10, 0)} м проти {fmt(L_opt, 0)} м); {opt2}
   ({fmt(L_op10, 0)} м, {fmt(gap_op)}% понад оптимум). Число мене не
здивувало: останній перехід жадібного
   маршруту назад на вокзал — {fmt(last_leg, 0)} м ({fmt(100 *
last_leg / L_nn10, 0)}% усієї довжини), бо зручні сусіди вже
```

```
   витрачені. Картограма H3 пояснює чому: половина всіх кандидатів
лежить у {half} з {len(dens)} зайнятих комірок,
   тобто маршрут — щільна петля в центрі плюс виїзди до кількох
віддалених точок, які жадібний метод лишає на кінець.
   На {len(ids30)} точках 2-opt скоротив маршрут найближчого сусіда
   з {fmt(L_nn30, 0)} до {fmt(L_op30, 0)} м (на {fmt(100 * (1 - L_op30
/ L_nn30), 1)}%) за {fmt(t_op30, 3)} с, але відстань до оптимуму
   невідома: гарантій немає, порівнювати можна лише методи між собою.

6. Таблиця Held-Karp для {len(ids30)} точок має 2^{len(ids30) - 1} *
{len(ids30) - 1} = {sci(cells30)} клітин;
   при 8 байтах на клітину це ~{cells30 * 8 / 1e9:.0f} ГБ пам'яті і
~{sci(ops30)} операцій. Таблиця не вміщається
   в пам'ять ноутбука: це питання обсягу пам'яті, а не часу. Кожні
додаткові 5 точок множать потребу приблизно у 30 разів.

7. Що виявилось неправильним у даних:
   - дублікати: у «сирому» топ-30 {len(pairs)} пар ближче 60 м; точних
збігів назви в кандидатах - {dup_names} (наприклад «Євро-Арт»,
     два об'єкти за ~12 м). Правило «лишаю вищого за рейтингом»
відкинуло {len(dropped)} об'єктів, {d_top30} із них зачепили топ-30.
     Правило не розрізняє справжній дублікат і два різні об'єкти поруч
(група танків біля меморіалу, музей і пам'ятник біля нього)
     і вирішує лише за рейтингом; тому рейтинг мусить бути змістовним.
Відкинуті музеї: {', '.join(lost_museums) if lost_museums else
'немає'};
   - назви: помилки (наприклад «Чорнобильскої», «Австостанція» у шарі
станцій), англійська назва «Historical map of Rivne»
     в українському шарі, кілька назв - не пам'ятки («Фотозона Я ❤
Рівне», «VW T4 (уламки)»);
   - прив'язка до графа: мала задача - {len(set(ids10.tolist()))}
різних вузлів на {len(ids10)} точок, велика -
{len(set(ids30.tolist()))} на {len(ids30)}
     (злиплись груп: {len(glued10)} і {len(glued30)}), тож нульових
віддалей між різними точками немає;
   - безіменні об'єкти: {N_NONAME} відкинуто (питання 1);
wikidata/wikipedia є лише у {n_wiki} з {N_CAND}, з них у меморіалів
{n_wiki_mem}.

Крок 1 (два способи даних). {cost_txt}

Декларація про використання штучного інтелекту.
Користувався ШІ-помічником (Claude, Anthropic): написання й правки
комірок, чернетка тексту звіту. Перевірки в ноутбуці:
метричні розрахунки лише після to_crs(32635); у H3 і shapely порядок
(lng, lat) - середнє ребро комірки ~201 м; close_pairs
перевірено на завідомо близьких точках; структура маршрутів
(check_tour) і нерівність Held-Karp <= 2-opt <= найближчий сусід;
довжина ламаної на карті звіряється з матрицею; osmnx 2.x
(features_from_*, без застарілого geometries_from_*);
```

```
порівняння з рейтингом за площею
виправлено після того, як з'ясувалось, що площа визначена лише для
полігонів. Висновки про ваги, пам'ятки й дані я
перечитав і погодив із виводами комірок вище.
'''
print(REPORT)


ЗВІТ · Перхун Максим, № 11 · місто: Рівне, Україна
Дані © OpenStreetMap contributors, ліцензія ODbL. Знімок даних надано
викладачем.

1. Відкинуто 14 з 119 об'єктів (12%) за єдиним правилом: немає тега
name
   (маршрут не може містити «якусь історичну будівлю»). Серед
відкинутих 5 оглядових точок і 5 меморіалів:
   туристові вони могли б знадобитись, але без назви їх неможливо
вказати в маршруті. За тегами відкинуто 0, бо запит
   повертав лише historic/tourism. Фільтр без пояснень я б замовнику
не віддав: із 105 кандидатів 49 (47%)
   - меморіали (пам'ятні знаки), ще 9 - військова техніка на
постаментах (танки, гармати, літак), 24 - artwork
   (скульптури, муралі, фотозони) і лише 3 - музеїв. Тобто історичні й
туристичні теги в OSM охоплюють значною мірою меморіальні
   знаки та вуличні об'єкти (78% кандидатів: меморіали, техніка,
artwork), а не історико-культурні об'єкти в
   туристичному сенсі; без цього застереження замовник сприйняв би
список як перелік головних атракцій.

2. Сигнали й ваги: wikidata/wikipedia 0.35; кількість мов name:*
(ранг, 0 лишається 0) 0.25;
   tourism 0.15; historic 0.15; площа будівлі (ранг) 0.1. Сигнали
visit і hist градуйовані:
   музей, галерея, замок, вежа = 1; пам'ятник = 0,8; меморіал = 0,5;
скульптура = 0,4; військова техніка = 0,2.
   Бінарний historic ставив би танк на постаменті нарівні із замком.
Міркування щодо ваг: найбільшу вагу має
   незалежне визнання (стаття у Вікіданих), бо його ставить хтось поза
OSM; кількість перекладів - ознака інтересу
   іноземців; тип об'єкта (visit, hist) - груба класифікація, тому
вага менша; площа найслабша (великим буває й склад)
   і тут майже не працює: ненульова лише для 4 з 105 кандидатів, решта
- точки. Обмеження: wiki мають
   лише 7 з 105, тож вага 0,35 діє на невелику частку об'єктів. Для
меморіалів wikidata імовірно
   стосується людини, якій присвячено пам'ятник, а не самого знаку,
тож сигнал частково міряє відомість людини.
   Ваги обирав міркуванням, не підганяв під результат.

3. За площею рейтинг визначений лише для 4 полігонів (решта мають
```

площу 0), тож порівняти можна 4
   об'єктів, а не 10. У моєму топ-10 збіглося 2 з 4: Басівкутське городище, Вхідна вежа. Перший за моїм критерієм:
   Таємничі підземелля Рівного; перший за площею: Басівкутське городище. Ті самі дані, той самий код,
   без помилок - і різні списки. «Найкращі» - не властивість даних, а наслідок критерію, за який відповідає аналітик.

4. Коефіцієнт обходу: 1,22 на малій задачі і 1,20 на великій - на нижній межі типового інтервалу 1,2-1,6. Пояснення: мережа піша
   (34% ребер - footway/path/steps/pedestrian), пішохідні доріжки й проходи між кварталами скорочують шлях,
   тому дорога лише на ~20% довша за пряму. Недосяжних пар 0: попри річку й залізницю, у місті
   достатньо переходів, і коефіцієнт не роздутий. Асиметрія малої матриці: макс. 0 м - для пішої
   мережі очікувано. Точки сіли на вузли в середньому за 21 м (макс. 51 м), тож
   похибка прив'язки мала.

5. Найближчий сусід гірший за точний розв'язок на 1,95% (14597 м проти 14317 м); 2-opt лишився гіршим за оптимум на 1,95%
   (14597 м, 1,95% понад оптимум). Число мене не здивувало: останній перехід жадібного
   маршруту назад на вокзал — 4320 м (30% усієї довжини), бо зручні сусіди вже
   витрачені. Картограма НЗ пояснює чому: половина всіх кандидатів лежить у 8 з 43 зайнятих комірок,
   тобто маршрут — щільна петля в центрі плюс виїзди до кількох віддалених точок, які жадібний метод лишає на кінець.
   На 31 точках 2-opt скоротив маршрут найближчого сусіда
   з 35358 до 30417 м (на 14,0%) за 0,009 с, але відстань до оптимуму
   невідома: гарантій немає, порівнювати можна лише методи між собою.

6. Таблиця Held-Karp для 31 точок має 2^30 * 30 = 3,2*10^10 клітин;
   при 8 байтах на клітину це ~258 ГБ пам'яті і ~9,7*10^11 операцій. Таблиця не вміщається
   в пам'ять ноутбука: це питання обсягу пам'яті, а не часу. Кожні додаткові 5 точок множать потребу приблизно у 30 разів.

7. Що виявилось неправильним у даних:
   - дублікати: у «сирому» топ-30 9 пар ближче 60 м; точних збігів назви в кандидатах - 4 (наприклад «Євро-Арт»,
     два об'єкти за ~12 м). Правило «лишаю вищого за рейтингом» відкинуло 43 об'єктів, 8 із них зачепили топ-30.
     Правило не розрізняє справжній дублікат і два різні об'єкти поруч (група танків біля меморіалу, музей і пам'ятник біля нього)
     і вирішує лише за рейтингом; тому рейтинг мусить бути змістовним. Відкинуті музеї: Краєзнавчий музей, Музей бурштину;
   - назви: помилки (наприклад «Чорнобильскої», «Австостанція» у шарі

станцій), англійська назва «Historical map of Rivne»
     в українському шарі, кілька назв - не пам'ятки («Фотозона Я ❤ Рівне», «VW T4 (уламки)»);
   - прив'язка до графа: мала задача - 11 різних вузлів на 11 точок, велика - 31 на 31
     (злиплись груп: 0 і 0), тож нульових віддалей між різними точками немає;
   - безіменні об'єкти: 14 відкинуто (питання 1); wikidata/wikipedia є лише у 7 з 105, з них у меморіалів 6.

Крок 1 (два способи даних). Живого запиту не виконувалось (Overpass недоступний); порівняння за часом і обсягом не наводжу.

Декларація про використання штучного інтелекту.
Користувався ШІ-помічником (Claude, Anthropic): написання й правки комірок, чернетка тексту звіту. Перевірки в ноутбуці:
метричні розрахунки лише після to_crs(32635); у H3 і shapely порядок (lng, lat) - середнє ребро комірки ~201 м; close_pairs
перевірено на завідомо близьких точках; структура маршрутів
(check_tour) і нерівність Held-Karp <= 2-opt <= найближчий сусід;
довжина ламаної на карті звіряється з матрицею; osmnx 2.x
(features_from_*, без застарілого geometries_from_*);
порівняння з рейтингом за площею
виправлено після того, як з'ясувалось, що площа визначена лише для полігонів. Висновки про ваги, пам'ятки й дані я
перечитав і погодив із виводами комірок вище.